

Практическая работа «Восстановление изображений в системе MATLAB»

Цель: получение навыков формирования аддитивных шумов в системе Matlab. Обучение восстановлению изображения после шумового воздействия.

Содержание отчета: ответы на контрольные вопросы, выполненные задания и вывод о проделанной работе.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Зашумление изображений

В процессе фотосъемки некоторые изображения подвергаются шумовому воздействию. Поэтому обработка графических файлов неотделима от операции восстановления их приблизительного первоначального вида путем устранения шумов различного типа. Искажение изображения в общем виде представлено в виде формулы [1]:

$$g(x,y) = H[f(x,y)] + n(x,y) \quad (1)$$

где $g(x,y)$ – искаженное изображение, H – искажающий оператор, $n(x,y)$ – аддитивный шум.

Общая схема искажения/восстановления изображения представлена на рисунке 3.1 [1].

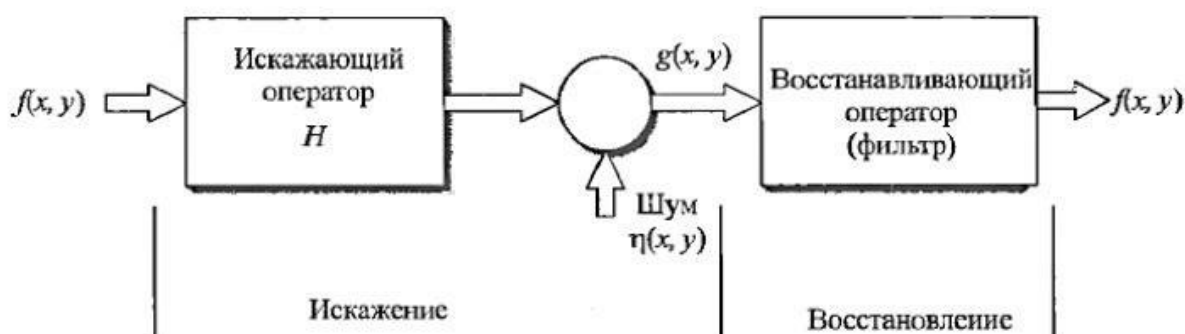


Рисунок 3.1. – Моделирование процесса искажения/восстановления изображения

Как видно из рисунка 3.1, искажение изображения осуществляется за счет искажающего оператора и аддитивного шума, восстановление изображения – за счет процедуры фильтрации.

Аддитивный шум подразделяется на два типа [1]:

- пространственный – характеризуется функцией плотности вероятности (PDF, Probability Density Function) или функцией кумулятивного распределения (CDF, Cumulative Distribution Function);
- частотный – описываемый различными частотными характеристиками преобразования Фурье.

В данной лабораторной работе искажения вносятся только пространственным шумом. В качестве графического файла выбрано изображение полутонового типа.

Формирование аддитивного пространственного шума

В пакете IPT системы Matlab добавление шума в изображение осуществляется с помощью функции *imnoise*, имеющей вид:

$$g = \text{imnoise}(\text{image}, \text{noiseType}, \text{parameters})$$

где *image* – исходное изображение, *noiseType* – тип шума, *parameters* – параметры шума, *g* – зашумленное изображение.

Функция *imnoise* поддерживает следующие типы шумов:

- Гауссов шум – задается параметром *'gaussian'*
- Локальный гауссов шум - *'localvar'*
- Шум Пуассона - *'poisson'*
- Шум типа соли и перца – *'salt & pepper'*
- Спекл-шум (мультипликативный шум) – *'speckle'*

Пример:

В данном примере исходное изображение подвергнем зашумлению шумом Гаусса с нулевым средним и дисперсией 0.1. Исходное изображение представлено на рисунке 3.2(а), зашумленное – на рисунке 3.2(б).

```
image = imread('cameraman.tif');
```

```
image = im2double(image); %Все значения яркости изображения  
закljučаются в диапазон [0; 1]
```

%В данной работе подобная операция обязательна

```
noisedImage = imnoise(image, 'gaussian', 0, 0.1);
```

```
imshow(image), figure, imshow(noisedImage)
```



а



б

Рисунок 3.2

а) Исходное изображение,

б) Зашумленное изображение

Функция *imnoise* удобна для зашумления изображения. Однако если нам нужна матрица самого шума, то его необходимо реализовывать самостоятельно.

Пространственный шум основан на использовании генератора случайных чисел. В Matlab встроены две функции, отвечающие за генерацию случайных чисел [1]:

- *rand* – генерирует матрицу равномерно распределенных случайных величин в интервале (0, 1).
- *randn* – генерирует матрицу с независимыми гауссовыми (нормальными) случайными величинами с нулевым средним и единичной дисперсией.

Функция *rand* имеет следующий синтаксис:

$numbers = rand(M, N)$

где *numbers* – матрица случайных чисел, *M* и *N* – размер матрицы. Если не указаны параметры *M* и *N*, то генерируется скалярная случайная величина. Если указана только одна сторона матрицы, то она считается квадратной.

Синтаксис функции *randn* аналогичен синтаксису функции *rand*:

$numbers = randn(M, N)$

Создание генераторов различных пространственных шумов основано на использовании данных функций.

В таблице 3.1 представлены виды пространственных шумов и генераторы, их реализующие.

Пример:

Рассмотрим реализацию гауссова шума.

```
noise = randn(256, 256);
```

```
imshow(noise)
```

В данном примере был создан гауссов шум в виде матрицы размером 256x256 элементов. Графическое изображение шума представлено на рисунке 3.3.

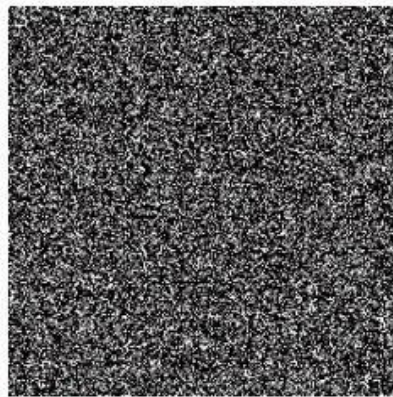


Рисунок 3.3. – Гауссов шум размером 256x256

Восстановление изображений

Восстановление изображений осуществляется при помощи фильтрации. В предыдущей практической вы уже использовали фильтрацию для выделения границ объектов изображения. В данной работе фильтрация используется для устранения шума. Напомним принцип работы фильтров [1]:

Фильтр представляет маску размером 3×3, которая в режиме построочного сканирования скользящим окном проходит по всем пикселям изображения. Данный процесс отображен на рисунке 3.4 [2]. Если матрица пикселей выходит за пределы изображения, то ее выходящие за пределы изображения пиксели считаются равными нулю (так называемый граничный эффект).

Полученная на каждом шагу матрица суммируется по представленной ниже формуле [2]:

$$R = w(-1,-1)f(x-1, y-1) + w(-1,0)f(x-1, y) + \dots + w(0,0)f(x, y) + \dots + w(1,0)f(x+1, y) + w(1,1)f(x+1, y+1)$$

где R – область изображения, подвергнутая фильтрации.

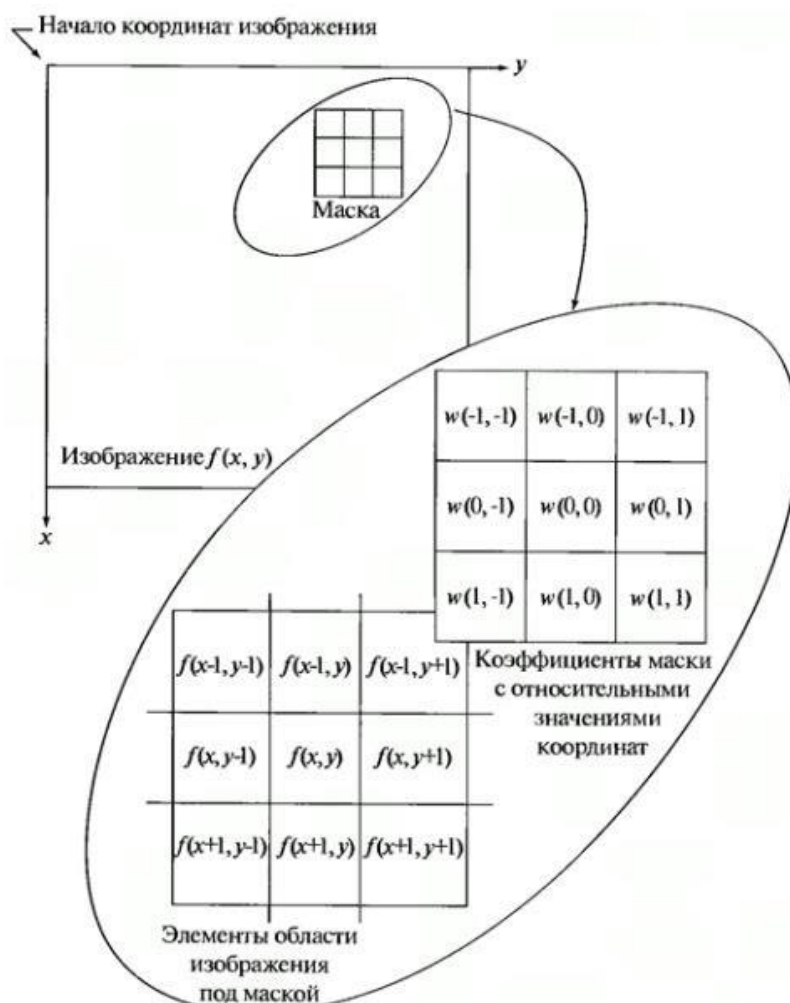


Рисунок 3.4. – Принцип работы фильтра изображения.

В таблице 3.2 представлены различные типы фильтров, предназначенных для фильтрации изображения. Каждый фильтр обладает различной мерой эффективности против конкретного вида шумового воздействия.

В системе Matlab встроены функции реализации фильтров, но только некоторых.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Задание № 1:

Используя данные таблицы 1, создайте функции, реализующие каждый из представленных видов шумов. Программный код функций с комментариями поместите в отчет.

Таблица 3.1.

Тип шума	Генератор шума (z)	Параметры для данной работы	
		a	b

Равномерный	функция <code>rand</code>	-	-
Гауссов	функция <code>a + b*randn</code>	0	0.15
Логарифмически нормальный	$z = a * e^{b \cdot N(0,1)}$	1	0.25
Реллея	$z = \sqrt{b \cdot \ln \cdot [1 - U(0,1)]}$	0	1
Экспоненциальный	$z = -\frac{1}{a} \cdot \ln[1 - U(0,1)]$	1	-
Эрланга	$z = E_1 + E_2 + \dots + E_b$, где $E_n = -\frac{1}{a} \cdot \ln[1 - U(0,1)]$	2	5

$N(0, 1)$ – нормальная Гауссова величина со средним 0 и дисперсией 1 (аналогична функции `randn`). $U(0, 1)$ – равномерная случайная величина из интервала (0, 1) (аналогична функции `rand`).

Построим гистограмму шума Гаусса из предыдущего примера. Для создания гистограммы в системе Matlab предусмотрена функция `hist`, которая имеет следующий синтаксис [1]:

$$histogram = hist(r, bins)$$

где `histogram` – полученная гистограммы, `r` – исходный массив данных, `bins` – количество корзин гистограммы.

Таким образом, гистограмму данного шума можно построить следующим образом (рисунок 3.5)

Пример:

`noise = randn(256, 256);`

`hist(noise(:), 50)`

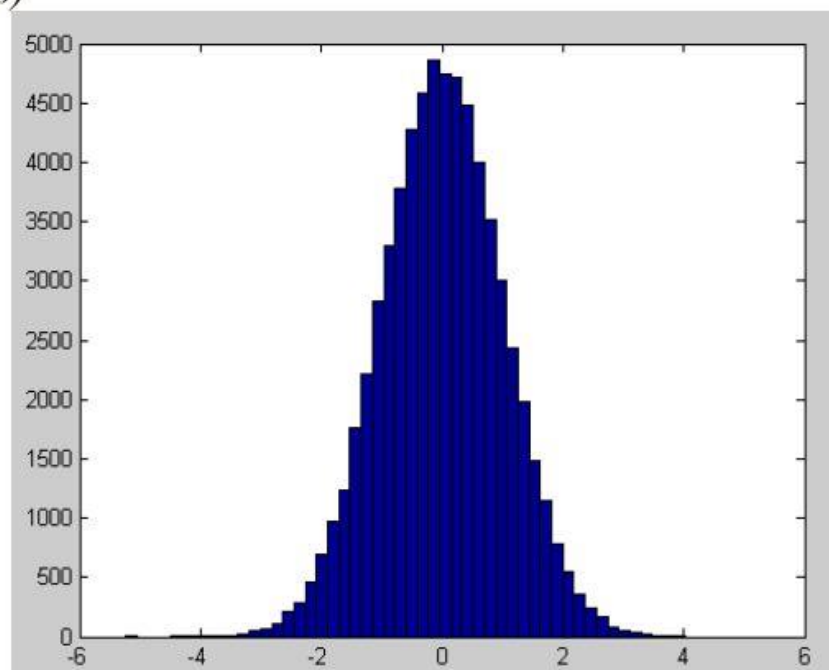


Рисунок 3.5. – Гистограмма Гауссова шума.

Задание № 2:

Осуществите генерацию каждого типа шума размером 256×256 при помощи созданных функций. Для всех шумов, кроме шума Соль и перец, создайте гистограмму и поместите их в отчет. Количество корзин гистограммы должно иметь значение, равное 50.

Используя матрицу шума, можно легко осуществить зашумление изображения. Зашумление осуществляется путем сложения матриц пикселей изображения и матрицы шумовых компонент:

$$\text{noisedImg} = \text{image} + \text{noise}$$

где *noisedImg* – зашумленное изображение, *image* – исходное изображение, *noise* – шум.

Пример:

```
image = imread('cameraman.tif');  
image = im2double(image);  
noise = randn(256);  
noisedImg = image + noise;  
imshow(noisedImg)
```

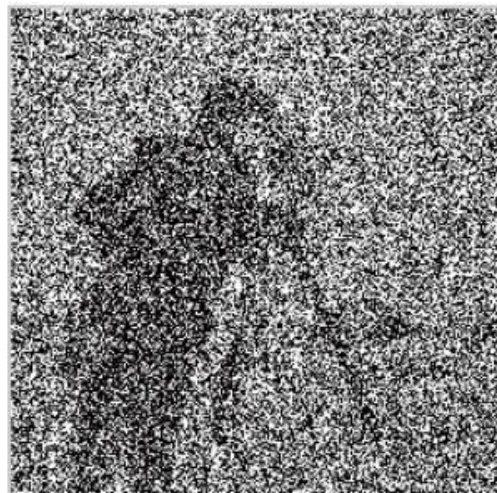


Рисунок 3.6. – Изображение, подвергнутое зашумлению шумом Гаусса

Задание № 3:

Используя созданные функции и выбранное для данной практической работы изображение, осуществите зашумление полученного изображения при помощи всех видов шумов, представленных в таблице 1. Каждый полученный файл сохраните в виде отдельного изображения. Полученные изображения для каждого вида шума поместите в отчет, указав при этом вид шума, которому изображение соответствует.

Задание № 4:

Создайте функции, реализующие фильтрацию изображения каждым фильтром, формула которого представлена в таблице 2. При реализации конкретного фильтра используйте только его формулу. Код программы с комментариями поместите в отчет.

В таблице 3.2 n и m – разрешение окна фильтрации. В качестве окна фильтрации используйте окно размером 3×3 . G – исходное зашумленное изображение. S и t – координаты середины окна просмотра.

Таблица 3.3.

Имя фильтра	Уравнение
Арифметическое среднее	$\hat{f}(x, y) = \frac{1}{mn} \sum_{(s,t) \in S_{xy}} g(s, t)$
Геометрическое среднее	$\hat{f}(x, y) = \left[\prod_{(s,t) \in S_{xy}} g(s, t) \right]^{\frac{1}{mn}}$
Гармоническое среднее	$\hat{f}(x, y) = \frac{mn}{\sum_{(s,t) \in S_{xy}} \frac{1}{g(s, t)}}$
Контрагармоническое среднее	$\hat{f}(x, y) = \frac{\sum_{(s,t) \in S_{xy}} g(s, t)^{Q+1}}{\sum_{(s,t) \in S_{xy}} g(s, t)^Q}$
Медиана	$\hat{f}(x, y) = \text{median}_{(s,t) \in S_{xy}} \{g(s, t)\}$
Максимум	$\hat{f}(x, y) = \max_{(s,t) \in S_{xy}} \{g(s, t)\}$
Минимум	$\hat{f}(x, y) = \min_{(s,t) \in S_{xy}} \{g(s, t)\}$
Срединная точка	$\hat{f}(x, y) = \frac{1}{2} \left[\max_{(s,t) \in S_{xy}} \{g(s, t)\} + \min_{(s,t) \in S_{xy}} \{g(s, t)\} \right]$
α -усеченное среднее	$\hat{f}(x, y) = \frac{1}{mn-d} \sum_{(s,t) \in S_{xy}} g_r(s, t)$

Задание № 5:

- Каждое изображение, полученное в задании 3, подвергните фильтрации всеми фильтрами, используя созданные в задании № 4 функции. Используя функцию корреляции Пирсона `corr2`, определите качество каждого восстановленного изображения, сравнив его с оригиналом. Код программы с комментариями поместите в отчет.

- Выполняя данное задание, заполните таблицу 3.4 и поместите ее в отчет. В каждую ячейку, находящуюся на пересечении строки с типом шума и столбца с типом фильтра необходимо вписать значение корреляции между оригиналом и восстановленным изображением. **Жирным** выделить значение наилучшего показателя корреляции Пирсона.

Таблица 3.4

	Равномерный	Гауссов	Логарифмически нормальный	Реллея	Экспоненциальный	Эрланга
Арифметическое среднее						
Геометрическое среднее						
Гармоническое среднее						
Контргармоническое среднее						
Медиана						
Максимумы						
Минимумы						
Срединная точка						
α -усеченное среднее						

Сделайте вывод об эффективности каждого фильтра для конкретного вида шума.

Перечень контрольных вопросов

- 1- В чем заключается процесс искажения/восстановления изображения?
- 2- На какие типы делятся шумы, вносимые в изображения?
- 3- На чем основаны пространственные фильтры?
- 4- Приведите примеры линейных пространственных фильтров.
- 5- Приведите примеры нелинейных пространственных фильтров.
- 6- Какие помехи и шумы устраняет медианный фильтр?